

Forget Without Seeing: Privacy-Preserving Subliminal Unlearning for Post-Trained Vision-Language Models

Anonymous Authors

Abstract—Federated unlearning aims to remove the influence of deleted data from a federated model while preserving the privacy benefits of federated learning (FL). Existing unlearning privacy attacks show that the unlearning process itself may leak information about forgotten data, but they commonly rely on sample-level gradients or client updates. These assumptions are difficult to satisfy in privacy-preserving federated learning systems with secure aggregation. Under secure aggregation, the curious FL server cannot obtain any gradients or individual updates, but only the aggregated models.

In this paper, we ask whether secure aggregation is sufficient to hide what has been forgotten. We propose CrossLeak, which treats unlearning privacy leakage as a paired-model comparison problem. By probing the original and unlearned aggregated models with the same public samples, it filters out stable model behavior and highlights representation directions that are weakened by deletion. A sparse CrossCoder then separates shared features from unlearning-sensitive features, enabling CrossLeak to infer deleted labels from representation and confidence shifts and to recover deletion-sensitive concepts from before-exclusive latent features. This design avoids the need for sample-level gradients or local updates, making the attack applicable even under secure aggregation.

We evaluate CrossLeak on representative datasets and federated unlearning methods, including FedU, Hessian-FU, and exact federated retraining. The results show that even after secure aggregation hides client updates and gradients, the before-after difference between aggregated models can still reveal deleted labels and deletion-sensitive visual concepts. These findings expose a new privacy risk in federated unlearning: secure aggregation protects client updates, but it does not necessarily hide the information leaked by the act of forgetting itself.

Index Terms—federated unlearning, secure aggregation, privacy leakage, crosscoder.

I. INTRODUCTION

Federated Unlearning is an emerging research branch in machine unlearning to uphold the right to be forgotten. Federated Learning (FL) protects users’ private data by keeping raw training data on local clients while only exchanging model updates or aggregated parameters with a central server [1, 2, 3]. When a client later requests deletion, the system must remove the influence of the corresponding data from the trained global models [4, 5]. Since retraining from scratch is computationally expensive, recent federated unlearning methods therefore aim to efficiently update the global model so that the unlearned model behaves as if the deleted data had never participated in training [5, 6].

There are few studies that have noticed the privacy leakage issues in machine unlearning and federated unlearning. For ex-

ample, model inversion style attacks attempt to reconstruct or characterize deleted samples [7], while label inference attacks try to infer which class labels were involved in the unlearning request [8]. These studies reveal an important tension: unlearning is designed to protect users by removing their influence, yet the act of removal may introduce a distinguishable model change that exposes what was removed [7, 8, 9, 10].

Research Gap. However, these unlearning privacy inversion methods highly rely on the sample-level unlearning gradients or intermediate updates. These information are hard to access in privacy-preserving federated unlearning with secure aggregation [11]. Secure aggregation [11, 12, 13] is a common privacy-preserving technique in FL to ensure that the FL server can only obtain the aggregated global model instead of individual local models and gradients. Under this deployment, a curious or malicious FL server may not observe client updates and gradients of the unlearned samples. The remaining attack surface is much more restricted: the FL server attacker can only compare the aggregated model before unlearning and the aggregated model after unlearning. It remains unclear whether such limited access is sufficient to infer sensitive information about the unlearning request.

Research Question. Given this state of the art, we pose the following research question: *“Given only access of the secure aggregated original and unlearned models, to what extent do current federated unlearning methods leak the sensitive information of unlearned data?”*

Inspiration. The key challenge is that secure aggregation limits the most direct attack signals: the adversary cannot observe client-level updates or sample-level gradients. Although the two aggregated models are available, all the updating and unlearning information is aggregated, making the deleted privacy inference significantly difficult. Inspired by CrossCoder [14, 15, 16], which provides a tool for analyzing paired model representations, we investigate whether unlearning privacy leakage can be inferred from the representation-space difference between the original model and the unlearned model.

Our Work. To answer the research question, we propose the CrossLeak attack to identify unlearning data leakage in privacy-preserving federated unlearning with secure aggregation, as depicted in Figure 1. CrossLeak is a post-unlearning attack that only requires access to the aggregated original model, the aggregated unlearned model, and a public probe set. It queries both models on the same probe samples, extracts

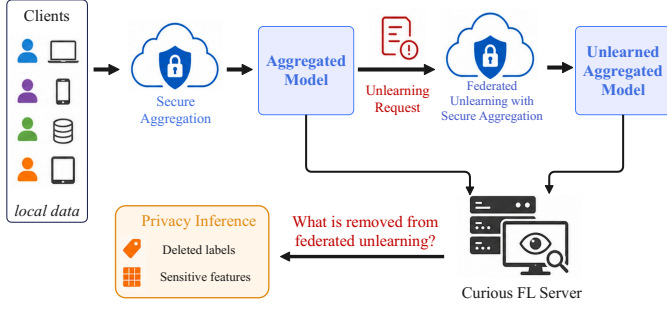


Fig. 1: Privacy leakage in secure-aggregation federated unlearning. Even when client-level updates and gradients are hidden, the aggregated original and unlearned models can still reveal information about the deleted labels and deletion-sensitive features.

paired representations and logits, and trains a sparse Cross-Coder to identify features that are active before unlearning but suppressed or shifted after unlearning. CrossLeak then uses these features, together with representation displacement and confidence degradation, to infer both deleted labels and deletion-sensitive visual concepts.

Feature Leakage. CrossLeak first studies leakage at the feature level. The key observation is that unlearning does not only affect final predictions; it also changes internal representation geometry. Features that are strongly supported by the deleted data may have high before-exclusive strength, large before-after decoder discrepancy, and high coverage on probe samples associated with the inferred deleted labels. By ranking these features, CrossLeak can recover deletion-sensitive latent evidence. For image tasks, CrossLeak further maps top-activating probe samples to semantic concepts with CLIP, which allows the attacker to infer visual attributes, object parts, or trigger-like patterns related to the unlearning request.

Label Leakage. CrossLeak also studies leakage at the label level. Deleted labels can leave several complementary traces: the corresponding class representations may move after unlearning, and the original model’s confidence on the deleted classes may drop. Thus, by observing the representations and prediction difference of the probe set, the attacker can infer the label of the unlearned data.

We conduct comprehensive experiments on three representative datasets across three representative federated retraining methods. We also compared our CrossLeak with existing state-of-the-art unlearning privacy inversion methods on federated unlearning [7, 8], where these attacks all need the unlearning gradients. The results demonstrate that CrossLeak can infer the private information (including the deleted labels and sensitive features) under the strict constraints after secure aggregation, without accessing the individual model and unlearning gradients.

Our contributions are summarized as follows.

- We introduce a new privacy leakage problem in secure-aggregation federated unlearning: inferring deleted labels

and deletion-sensitive features using only the original and unlearned aggregated models, without client gradients and access to unlearned samples.

- We propose CrossLeak, a CrossCoder-based attack that decomposes paired pre- and post-unlearning representations into shared and unlearning-sensitive features, and uses these features to perform label inference, sensitive feature inference, and semantic concept recovery.
- We conduct extensive experiments on representative datasets and federated unlearning methods. The results show that even after secure aggregation, CrossLeak can still infer deleted labels and deletion-sensitive features of current federated unlearning.

II. RELATED WORK

A. Federated Unlearning

Federated Unlearning (FU) extends machine unlearning [17, 18, 19] to FL, where the data to be forgotten are distributed across clients and their influence is embedded into the global model through iterative aggregation [5, 20]. Existing FU studies can be broadly grouped into three lines. The first line focuses on efficient removal mechanisms that avoid full federated retraining, such as rapid retraining for realizing the right to be forgotten in FL [5], class-discriminative pruning for class-level removal [4], and local client unlearning followed by limited additional FL rounds [21]. The second line improves the practicality of FU under more constrained FL settings, including asynchronous federated unlearning and user-side influence approximation, which reduce the reliance on synchronous retraining or extensive participation from the remaining clients [6, 22]. The third line considers the trustworthiness of the unlearning process itself, with VeriFi highlighting the need to verify whether the requested data influence has been faithfully removed [23]. While these studies have advanced FU in terms of efficiency, system practicality, and verifiability, they mainly treat unlearning as a mechanism for privacy compliance, leaving the privacy risks introduced by the unlearning process itself less explored.

B. Privacy Leakage in Federated Unlearning

Although federated unlearning is designed to support data removal and privacy compliance, recent studies show that the unlearning process itself may create new privacy and security risks [10]. A key observation is that the discrepancy between the original model and the unlearned model can expose information about the forgotten data, making unlearning-induced model variations a potential attack surface [24]. In centralized machine unlearning, unlearning inversion attacks have shown that an adversary may recover sensitive feature and label information from the gradients of unlearning operations [7].

This concern also occurs in federated unlearning. Recent attacks have investigated whether forgotten information can be reconstructed or inferred in FU, including model inversion attacks that recover information about the unlearned data and label inference attacks that infer the labels of forgotten samples, clients, or classes from unlearning-induced parameter

variations [8, 10]. However, these federated unlearning targeted attacks highly relied on the unlearning gradients, which is not accessed in privacy-preserving federated learning with secure aggregation. Therefore, exploring the privacy leakage of federated unlearning under widely used FL privacy mechanisms, such as secure aggregation, is still an open problem.

III. PRELIMINARY AND THREAT MODEL

A. Federated Learning and Federated Unlearning

We consider a standard federated learning (FL) system [25, 26] with a central server and N clients. Each client i holds a private local dataset D_i , and the global training set is $D = \bigcup_{i=1}^N D_i$. Instead of sending raw data to the server, clients train local models and the server aggregates them to obtain a global model. We denote the original global model trained before any deletion request as $f_{\theta_o} = \mathcal{A}_{\text{FL}}(D)$, where $\mathcal{A}_{\text{FL}}$ denotes the federated training algorithm and θ_o is the learned parameter.

Federated unlearning aims to remove the influence of a deletion set $D_u \subset D = \bigcup_{i=1}^N D_i$ from the trained global model. Let $D_r = D \setminus D_u$ be the retained training data. Ideally, the unlearned model should behave like a model trained only on D_r : $f_{\theta_u} \approx \mathcal{A}_{\text{FL}}(D_r)$. Exact federated retraining obtains f_{θ_u} by retraining on D_r , while approximate federated unlearning methods update f_{θ_o} more efficiently to suppress the influence of D_u . In this paper, D_u may correspond to samples from one or more target labels, samples containing sensitive visual features, or backdoored samples with a trigger pattern.

B. Threat Model under Secure Aggregation

In privacy-preserving FL, secure aggregation hides individual client updates from the server and only reveals aggregated model information [11, 12, 13]. Thus, during training and unlearning, the server does not observe a client-specific gradient $\nabla \ell(D_i; \theta)$ and a local model update. This changes the privacy attack surface in federated unlearning.

To make the threat model convenient to understand, we assume that the FL server is an honest-but-curious attacker in a secure-aggregation federated unlearning system [27]. The attacker follows the protocol and does not tamper with training or unlearning, but tries to infer private information from the released aggregated models before and after unlearning. The key restriction is that the client-level signals, including local updates and gradients, remain hidden by secure aggregation [12].

Attacker’s capabilities. The attacker can access the released original model f_{θ_o} and the released unlearned model f_{θ_u} after secure aggregation. The attacker has a public probe set $D_p = \{x_j\}_{j=1}^n$, drawn from the same input domain. The probe set is disjoint from D_u and D . Probe labels may be available, but they are not required: if labels are unavailable, the attacker can use f_{θ_o} to assign pseudo-labels to probe samples. The attacker can query and inspect both f_{θ_o} and f_{θ_u} on the same probe samples. Thus, the attacker can obtain output probabilities, logits, and representations.

Attacker’s restrictions. Under the strict secure aggregation, the attacker cannot access raw client data, deleted samples, client gradients, and individual local updates. The attacker also does not know the deletion-request metadata, including which client requested unlearning, which sample indices were deleted, which labels were involved, or how many labels were deleted. Since the FL server is honest-but-curious, it will not poison training or alter the unlearning algorithm to assist unlearning privacy inference.

C. Attacking Goals of Unlearning-Induced Leakage

Secure aggregation hides client-level training signals, but the attacker can still observe the before-after aggregated model pair $(f_{\theta_o}, f_{\theta_u})$. We ask whether their difference leaks sensitive information about the deleted data D_u :

$$\mathcal{I}(D_u) \leftarrow \mathcal{T}(f_{\theta_o}, f_{\theta_u}, D_p), \quad (1)$$

where $\mathcal{T}$ is an attack algorithm and $\mathcal{I}(D_u)$ denotes private information about the unlearning request.

We focus on two types of private information that are actionable in federated unlearning.

Problem 1 (Deleted-Label Leakage). *Given the original model f_{θ_o} , the unlearned model f_{θ_u} , a public probe set D_p , and the label space $\mathcal{Y}$, the attacker aims to infer the deleted label set $Y_u \subseteq \mathcal{Y}$ associated with the unlearning request. The output is a ranked label list $\hat{Y}_u$.*

Problem 2 (Deletion-Sensitive Feature Leakage). *Given f_{θ_o} , f_{θ_u} , and a probe set D_p , the attack goal is to infer sensitive latent or semantic features that were supported by the deleted data D_u and weakened after unlearning. The attacker outputs a ranked feature set $\hat{F}_u$ that captures deletion-sensitive representation directions, and, when possible, a set of human-interpretable concepts $\hat{C}_u$ describing these features. The attack is successful when the recovered features and concepts are strongly associated with the deleted data D_u .*

The attacker is successful if $\hat{Y}_u$ overlaps with the true deleted label set Y_u , or if $\hat{F}_u$ captures the key features that are active before unlearning and suppressed or shifted after unlearning. This formulation captures a stricter setting than prior unlearning inversion attacks that require gradients and simulated deleted samples: the only private signal available to the attacker is the two secure aggregated models.

IV. THE CROSSLINK ATTACK

A. Overview

CrossLeak is a post-unlearning privacy attack that infers what has been removed from a federated model. The key observation is that unlearning does not merely change the final prediction scores. It also changes the internal representation geometry: directions that were strongly supported by the deleted data are weakened, displaced, or made less class-consistent after unlearning. These changes can remain visible even when the unlearning protocol hides client updates

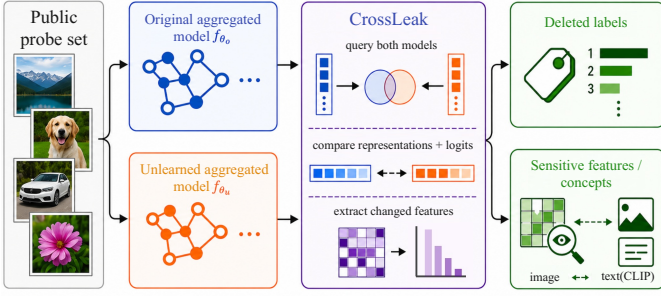


Fig. 2: Overview of the CrossLeak attack. CrossLeak compares the original and unlearned aggregated models on a public probe set, trains a sparse crosscoder on paired representations, and infers deleted labels and deletion-sensitive features from the crosscoder generated statistical differences.

through secure aggregation and even when the released model preserves normal test accuracy.

CrossLeak has two preparation stages. First, it collects paired pre- and post-unlearning representations from the two models on the same probe inputs. Second, it trains a sparse CrossCoder to decompose the paired representations into shared features and unlearning-specific changes. Then, CrossLeak scores labels by aggregating representation-level and prediction-level evidence. CrossLeak can also recover deletion-sensitive features and maps them to human-interpretable concepts using top-activating examples and CLIP-based concept matching. We demonstrate an overview process of CrossLeak in Figure 2.

B. Paired Model Signals

For each probe input $x_i \in D_p$, CrossLeak extracts both logits and intermediate representations:

$$g_i^o = f_{\theta_o}(x_i), \quad g_i^u = f_{\theta_u}(x_i), \quad (2)$$

$$z_i^o = h_{\theta_o}(x_i), \quad z_i^u = h_{\theta_u}(x_i), \quad (3)$$

Here $h_{\theta}(\cdot)$ is the representation layer used by the attack. CrossLeak trains the CrossCoder on paired representations and keeps logits as prediction-level evidence for label inference. CrossLeak constructs a paired representation vector

$$q_i = [\bar{z}_i^o; \bar{z}_i^u], \quad (4)$$

where $\bar{z}_i^o$ and $\bar{z}_i^u$ are normalized representations. The paired form is important: using only f_{θ_u} reveals what the final model knows, whereas CrossLeak needs the direction and magnitude of what changed.

C. Sparse CrossCoder for Unlearning Differences

The central component of CrossLeak is a sparse CrossCoder trained on paired representations. The CrossCoder learns a dictionary of latent features that jointly reconstructs the before- and after-unlearning signals while separating shared information from unlearning-sensitive information. Given q_i , the encoder produces a sparse code $a_i \in \mathbb{R}^m$:

$$a_i = \sigma(W_e q_i + b_e), \quad (5)$$

where W_e and b_e are the encoder weight matrix and bias, and $\sigma(\cdot)$ is a nonnegative activation. The decoder reconstructs the two representation views:

$$[\hat{z}_i^o; \hat{z}_i^u] = W_d a_i + b_d. \quad (6)$$

Here W_d and b_d are the decoder weight matrix and bias. We view W_d as two decoder blocks, W_d^o and W_d^u , which reconstruct the before- and after-unlearning representations, respectively.

CrossLeak trains the CrossCoder with reconstruction, sparsity, and change-awareness objectives:

$$\begin{aligned} \mathcal{L}_{cc} = & \frac{1}{n} \sum_{i=1}^n (\|\hat{z}_i^o - z_i^o\|_2^2 + \|\hat{z}_i^u - z_i^u\|_2^2) \\ & + \lambda_1 \frac{1}{n} \sum_{i=1}^n \|a_i\|_1 + \lambda_{\Delta} \frac{1}{n} \sum_{i=1}^n \|\widehat{\Delta z}_i - \Delta z_i\|_2^2, \end{aligned} \quad (7)$$

where $\Delta z_i = \bar{z}_i^o - \bar{z}_i^u$ and $\widehat{\Delta z}_i = \hat{z}_i^o - \hat{z}_i^u$. The first term makes the learned features faithful to the model representations, the second term encourages a compact set of interpretable features, and the third term forces the dictionary to preserve the representation displacement induced by unlearning.

After training, each feature j has a before-side decoder vector w_j^o and an after-side decoder vector w_j^u . CrossLeak measures the unlearning sensitivity of feature j by

$$s_j^{\Delta} = \|w_j^o - w_j^u\|_2 \cdot \frac{1}{n} \sum_{i=1}^n a_{ij}. \quad (8)$$

Features with large s_j^{Δ} are active on probe samples and have a strong before-after discrepancy, making them candidates for deletion-sensitive concepts.

D. Deleted Label Inference

CrossLeak infers deleted labels by combining two families of signals: representation displacement and confidence degradation. For each class c , let D_p^c be the probe samples labeled as class c . The representation displacement score is

$$S_{\Delta z}(c) = \frac{1}{|D_p^c|} \sum_{x_i \in D_p^c} \|\bar{z}_i^o - \bar{z}_i^u\|_2. \quad (9)$$

This score is high when the class manifold moves substantially after unlearning.

The confidence-drop score measures whether the original model's support for class c is selectively reduced:

$$S_{\text{conf}}(c) = \frac{1}{|D_p^c|} \sum_{x_i \in D_p^c} (p_{i,c}^o - p_{i,c}^u). \quad (10)$$

where $p_{i,c}^o = \text{softmax}(f_{\theta_o}(x_i, c))$ is the prediction probability of samples on class c . This score captures deletion evidence that appears at the output layer. It is especially useful when unlearning directly suppresses the target class prediction without causing a large global representation shift.

The attacker ranks labels using one of these heads or a normalized combination:

$$S_{\text{label}}(c) = \alpha S_{\Delta z}^*(c) + \beta S_{\text{conf}}^*(c), \quad (11)$$

where S^* denotes min-max normalization across labels. If the deletion budget k is known, CrossLeak outputs the top- k labels. Otherwise, it uses a score gap or threshold rule to determine the predicted deletion set.

E. Deletion-Sensitive Feature Inference

Beyond labels, CrossLeak identifies which latent features were most affected by unlearning. Here, a feature denotes a sparse crosscoder feature, i.e., a learned dictionary direction in the paired representation space, which may correspond to a visual attribute, texture, class-specific pattern, or trigger-like cue. A feature is considered deletion-sensitive if it is (i) active on probe samples, (ii) substantially changed between the before- and after-unlearning decoders, and (iii) mainly associated with samples from the inferred deleted labels. CrossLeak combines these requirements through a feature ranking score:

$$S_{\text{feat}}(j) = s_j^\Delta \cdot R_j^{\text{before}} \cdot C_j^{\text{target}}, \quad (12)$$

where

$$R_j^{\text{before}} = \frac{\|w_j^o\|_2}{\|w_j^o\|_2 + \|w_j^u\|_2 + \epsilon} \quad (13)$$

is the before-exclusive strength ratio. Here ϵ is a small positive constant used only for numerical stability when both decoder norms are close to zero. A larger R_j^{before} means that feature j has more mass in the before-unlearning decoder than in the after-unlearning decoder, which is consistent with deleted information being suppressed by unlearning. The target-label concentration term is

$$C_j^{\text{target}} = \frac{1}{|T_j|} \sum_{x_i \in T_j} \mathbb{I}[\hat{y}_i \in \hat{Y}_u] \quad (14)$$

where T_j is the set of top-activating probe samples for feature j , $\hat{Y}_u$ is the inferred deleted-label set, and $\hat{y}_i = \arg \max f_{\theta_o}(x_i)$ is the label assigned by the original model. The final sensitive feature set $\hat{F}_u$ is obtained by ranking $S_{\text{feat}}(j)$ and selecting the top features.

Semantic Concept Recovery. Latent features are informative but not immediately human-readable. CrossLeak interprets a sensitive feature j by collecting its top activating probe samples T_j and comparing them with a vocabulary of candidate concepts. For image tasks, CrossLeak uses a CLIP encoder to embed both the top activating images and textual prompts. Let $\phi_I(\cdot)$ be the CLIP image encoder and $\phi_T(\cdot)$ be the CLIP text encoder. For concept r , the semantic score is

$$S_{\text{clip}}(j, r) = \frac{1}{|T_j|} \sum_{x_i \in T_j} \cos(\phi_I(x_i), \phi_T(r)). \quad (15)$$

The highest scoring concepts are used as explanations of the recovered feature.

F. CrossLeak Attack Pipeline

The complete CrossLeak pipeline is summarized in Algorithm 1. The attack takes the original model f_{θ_o} , the unlearned model f_{θ_u} , and a public probe set D_p as input. It first

Algorithm 1: CrossLeak Attack

Input: Original model f_{θ_o} ; unlearned model f_{θ_u} ; probe set D_p ; prediction budget k ; concept vocabulary $\mathcal{V}$.
Output: Inferred deleted labels $\hat{Y}_u$; deletion-sensitive features $\hat{F}_u$; semantic concepts $\hat{C}_u$.

```

// (A) Collect paired model evidence
1 foreach  $x_i \in D_p$  do
2    $g_i^o \leftarrow f_{\theta_o}(x_i), \quad g_i^u \leftarrow f_{\theta_u}(x_i);$ 
3    $z_i^o \leftarrow h_{\theta_o}(x_i), \quad z_i^u \leftarrow h_{\theta_u}(x_i);$ 
4    $p_i^o \leftarrow \text{softmax}(g_i^o), \quad p_i^u \leftarrow \text{softmax}(g_i^u);$ 
5    $q_i \leftarrow [\bar{z}_i^o, \bar{z}_i^u];$ 

// (B) Train sparse CrossCoder on paired representations
6 Train the CrossCoder by minimizing  $\mathcal{L}_{cc}$  in Eq. (7);
7 foreach CrossCoder feature  $j$  do
8   Compute displacement score  $s_j^\Delta$  using Eq. (8);
9   Collect top-activating probe samples  $T_j$ ;

// (C) Deleted-label inference
10 foreach candidate label  $c \in \mathcal{Y}$  do
11   Compute  $S_{\Delta z}(c)$  using Eq. (9);
12   Compute  $S_{\text{conf}}(c)$  using Eq. (10);
13   Compute  $S_{\text{label}}(c)$  using Eq. (11);
14  $\hat{Y}_u \leftarrow \text{TopK}_{c \in \mathcal{Y}} S_{\text{label}}(c)$  with budget  $k$ ;

// (D) Deletion-sensitive feature inference
15 foreach CrossCoder feature  $j$  do
16   Compute  $R_j^{\text{before}}$  and  $C_j^{\text{target}}$  using Eqs. (13) and (14);
17   Compute  $S_{\text{feat}}(j)$  using Eq. (12);
18  $\hat{F}_u \leftarrow \text{TopK}_j S_{\text{feat}}(j)$ ;
19 foreach feature  $j \in \hat{F}_u$  do
20   Match  $T_j$  to concepts in  $\mathcal{V}$  using  $S_{\text{clip}}$  in Eq. (15);
21  $\hat{C}_u \leftarrow$  top matched concepts for  $\hat{F}_u$ ;
22 return  $\hat{Y}_u$ ,  $\hat{F}_u$ , and  $\hat{C}_u$ ;

```

constructs paired model evidence by querying both models on the same probe samples. The paired representations are used to train the sparse CrossCoder, while logits and probabilities are retained for prediction-level label inference.

After the CrossCoder is trained, CrossLeak performs two inference tasks. For deleted-label inference, it ranks labels using representation displacement and confidence drop, and outputs the top- k labels when the deletion budget is known. For deletion-sensitive feature inference, it ranks CrossCoder features according to their before-after displacement, before-exclusive strength, and target-label concentration. Finally, CrossLeak maps the top-ranked features to semantic concepts using CLIP-based concept matching, producing both the in-

ferred deleted labels and human-interpretable feature evidence.

V. EXPERIMENTS

A. Research Questions

We evaluate CrossLeak by answering the following research questions.

RQ1: How effectively does CrossLeak infer deleted labels and deletion-sensitive features compared with state-of-the-art unlearning privacy leakage methods?

RQ2: What are the contributions of CrossLeak’s key components, including the CrossCoder and different label inference heads, and how do parameter choices and evaluation metrics affect its performance?

RQ3: How well does CrossLeak generalize to other unlearning scenarios, such as backdoor-trigger recovery and label inference under different unlearning methods?

B. Experimental Setting

Datasets. We conduct experiments on three widely used image classification datasets: CIFAR-10 [28], CIFAR-100 [28], and Tiny-ImageNet [29]. These datasets cover different label spaces and visual complexities, from 10-class object recognition to fine-grained 100-class classification and 200-class large-scale recognition.

Models. We select the CNN, ViT and ResNet-18 model architectures in our experiments, where as the main model architecture is ResNet-18 on CIFAR10, CIFAR100, and Tiny-ImageNet. We train each original model for 40 epochs with a learning rate of $\eta = 0.0001$ and a mini-batch size of 200. All algorithms are implemented using Pytorch and are conducted on NVIDIA A100 GPUs.

Federated Unlearning Methods. We evaluate CrossLeak under both exact and approximate unlearning settings. For exact unlearning, we use federated retraining on the retained data as the gold-standard baseline, which removes the direct training contribution of deleted samples by training from scratch without them. For approximate unlearning, we consider representative federated unlearning methods, including FedU [6] and Hessian-based federated unlearning (Hessian-FU/HBFU) [5]. This selection allows us to examine whether privacy leakage arises only from approximate update procedures or also remains observable after stronger retraining-based removal.

Attack Baselines. We compare CrossLeak with the latest unlearning-targeted model inversion attack [7, 8, 10]. Since prior attacks mainly reconstruct or infer deleted samples from model gradients and outputs, we adapt their outputs to the same evaluation target by mapping recovered evidence to deleted labels and deletion-sensitive visual concepts when applicable. For unlearning labels inference, we include the [7] with confidence-drop and ULIA [8]. For sensitive feature inference, we just evaluate the CrossLeak because our setting is much more practical, only having the models after aggregation. By contrast, existing studies [7, 10] highly relied on the gradients for single unlearning samples, which are

inaccessible in privacy-preserving federated unlearning with secure aggregation.

Metrics. For deleted-label inference, we report label inference overlap, i.e., the fraction of true deleted labels that appear in the predicted deleted-label set. In our class-range experiments, the attacker does not know the number of deleted classes, but we report the top-k number of predicted labels, which is matched to the number of deleted target classes. For deletion-sensitive feature inference, we report the fraction of recovered features that overlap with erased-data-sensitive features, denoted as feature-overlap@ k , the before-exclusive strength ratio measuring how much recovered feature mass is associated with pre-unlearning information, and target-label feature coverage. For semantic interpretability, we also evaluate CLIP-based concept recovery using concept coverage.

C. RQ1: Privacy Leakage Effectiveness

Overall Setup. In the experiments, for each dataset and each unlearning method, we first train an original federated model f_{θ_o} on the full training set. We then construct an unlearning request by deleting samples associated with a target label set Y_u . To evaluate the impact of deletion scale, we vary the deletion ratio from 1% to 5% of the training data. To evaluate the impact of deletion diversity, we also vary the deleted class range, where $|Y_u|$ controls the number of target labels involved in the unlearning request. We use FedU, Hessian-FU, and exact federated retraining to obtain the post-unlearning model f_{θ_u} , and repeat the experiment over 5 random seeds.

We compare CrossLeak with direct post-hoc confidence drop based inference [7] and the unlearning label inference attack (ULIA) [8] baselines. ULIA scores labels by probing how the pre- and post-unlearning models change around candidate classes. We report two ULIA variants: *ULIA known-k*, which is given the true deletion budget $|Y_u|$ and returns exactly $|Y_u|$ labels, and *ULIA dynamic*, which selects labels using its score threshold without requiring the deletion budget. This separates leakage under an attacker who knows the deletion size from leakage under an attacker who only observes the two released models.

1) Label Inference Evaluation: We instantiate label inference as a post-unlearning set inference task. In the strict threat model, the attacker does not know the deletion budget $|Y_u|$. Therefore, CrossLeak produces a ranked list of candidate deleted labels, which can be converted into a predicted set $\hat{Y}_u$ using a score-gap rule. For controlled comparison, we also report a known-budget evaluation where the top- $|Y_u|$ labels are selected; this setting should be interpreted as an oracle-budget evaluation rather than the default attacker knowledge. ULIA known-k uses the same oracle deletion budget, while ULIA dynamic selects labels using its own threshold without knowing $|Y_u|$. We report the overlap between the predicted label set $\hat{Y}_u$ and the true deleted label set Y_u in Figure 3.

Results. Figure 3 shows that unlearning leaves measurable label-level traces in both CIFAR10 and CIFAR100, but the leakage pattern depends strongly on the unlearning algorithm.

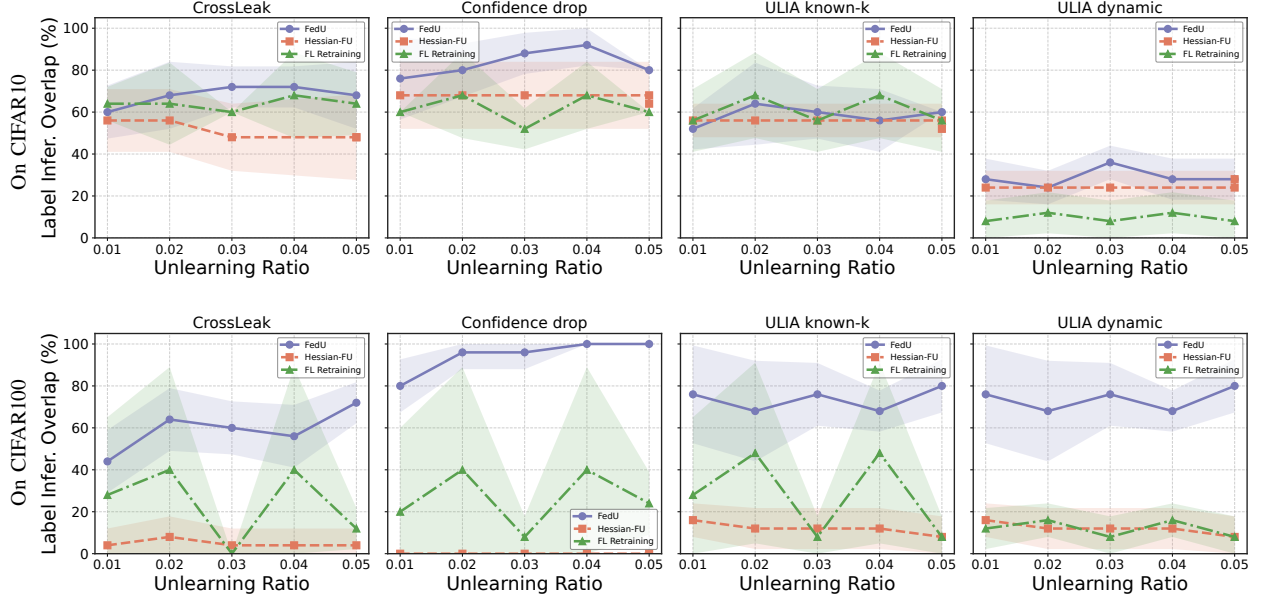


Fig. 3: Label Inference of Different Unlearning Ratio

Under FedU, the output-level confidence-drop signal is consistently strong because FedU directly suppresses information associated with the deleted classes. On CIFAR10, confidence drop achieves the highest overlap across most deletion ratios, while the representation-shift signal of CrossLeak also remains clearly above the dynamic ULIA baseline. On CIFAR100, the FedU leakage becomes even more pronounced: confidence drop nearly recovers the deleted label set at high deletion ratios, and ULIA known-k/dynamic are also competitive, indicating that class-specific removal in a larger label space can create a sharper post-unlearning fingerprint.

For Hessian-FU, the leakage is weaker and more dataset-dependent. On CIFAR10, confidence drop and CrossLeak still recover a non-trivial fraction of deleted labels, suggesting that approximate second-order unlearning leaves detectable class-wise changes even when the model utility is preserved. On CIFAR100, however, all label-inference heads drop substantially for Hessian-FU, showing that the same unlearning update can be much harder to exploit when each class has fewer samples and the class-wise representation shift is more diffuse. Exact retraining also produces weaker label leakage than FedU in most settings, especially on CIFAR100, which is consistent with retraining removing deleted samples without creating a localized update artifact.

2) *Sensitive Feature Inference Evaluation*: We evaluate sensitive feature inference by asking whether CrossLeak can recover latent features and semantic concepts that are specifically affected by the unlearning request. After training the CrossCoder, we rank features using their before-after decoder discrepancy, their before-exclusive strength, and their coverage on probe samples associated with the inferred deleted labels. A high before-exclusive strength indicates that the feature has

stronger mass in f_{θ_o} than in f_{θ_u} , while high target feature coverage indicates that the feature is concentrated on the target deleted labels rather than on unrelated retained labels. For semantic recovery, we map top-activating probe samples of recovered features to CLIP concepts and report CLIP concept coverage, together with before-exclusive strength and target feature coverage in Figure 5.

Previous unlearning model inversion attacks [7] rely on sample-level unlearning gradient to recover deleted information. This assumption does not directly match our federated threat model, where the curios server only compares the pre- and post-unlearning models, which are released by secure aggregation. The FL server does not access deleted samples or client-side gradients. Therefore, Figure 5 focuses on the sensitive feature inference achieved by CrossLeak under this more restrictive model-comparison setting.

Results. Figure 5 shows that CrossLeak can recover deletion-sensitive evidence beyond deleted labels. On CIFAR10, both FedU and Hessian-FU exhibit high target feature coverage across deletion ratios, indicating that the recovered CrossCoder features are consistently concentrated on the deleted-label region rather than on unrelated retained classes. Their CLIP concept coverage also remains high, showing that the recovered latent features correspond to visually coherent concepts that can be interpreted through top-activating samples. Exact retraining shows weaker before-exclusive strength and lower CLIP coverage, which is expected because retraining removes the deleted samples without applying a localized unlearning update that leaves a strong before-after feature artifact.

On CIFAR100, the leakage pattern is more method-dependent. FedU produces the strongest before-exclusive strength, close to saturation across deletion ratios, which

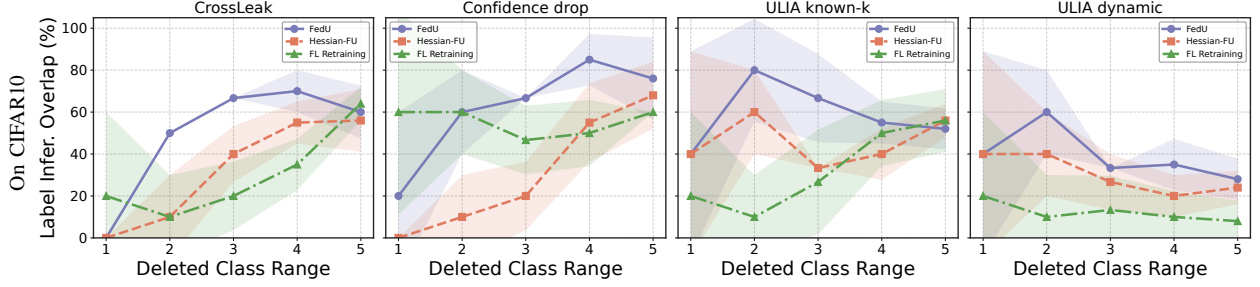


Fig. 4: Label Inference of Different Deleted Class Ranges

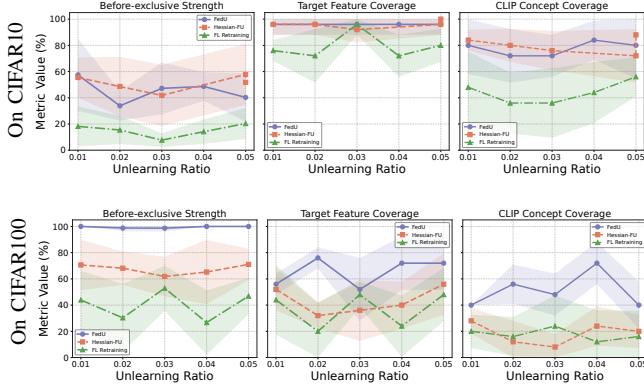


Fig. 5: Sensitive Feature Inference of Different Ratio

suggests that FedU creates highly separable pre-unlearning feature mass for the removed information. Its target feature coverage and CLIP concept coverage are also higher than those of Hessian-FU and retraining. In contrast, Hessian-FU and retraining still leave measurable feature traces, but their semantic recovery is weaker because CIFAR100 has a larger label space and fewer examples per class, making the top-activating features less visually concentrated. Overall, the results demonstrate that CrossLeak does not only infer which labels were deleted; it can also expose what latent and semantic features were affected by the unlearning request, even without gradients or deleted samples.

D. RQ2: Ablation Study of Different Variables

Overall Setup. RQ2 studies how the variables influence CrossLeak’s leakage signal, making it stronger or weaker. We conducted ablation studies for two key parameters, the *deleted class range* and the number of *unlearning clients*, where the *deleted class range* means the number of target labels included in the unlearning request. We use the same paired-model setting as RQ1, and all curves are averaged over 5 random seeds.

1) *Influence of Unlearning Range:* Figure 4 shows that the unlearning range has a clear effect on deleted-label inference. On CIFAR10, when the request contains only one deleted class, direct representation and output shifts are weak because

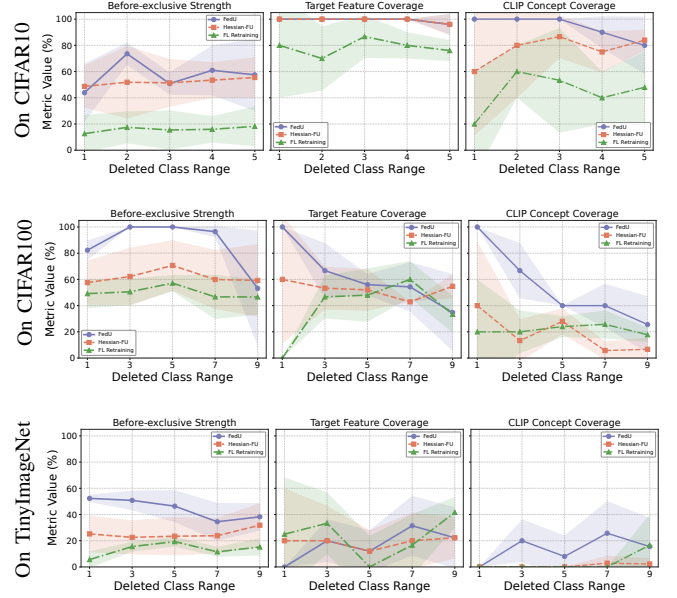


Fig. 6: Sensitive Feature Inference of Different Deleted Ranges

a single-class deletion produces a small aggregate change in the model. As the deleted class range increases, the signal becomes more separable: under FedU, confidence drop increases from about 20% overlap at range 1 to about 76% at range 5, while the Δz head increases from nearly 0% to about 60%. CrossLeak’s feature-vote head also improves with the range, reaching about 48% overlap at range 5. This trend indicates that a broader unlearning request leaves a more stable label-level fingerprint across probe samples.

The comparison across inference heads also shows that different heads capture different leakage sources. Confidence drop is the strongest head for FedU because FedU produces visible output-level suppression for deleted labels. ULIA known-k remains competitive when it is given the deletion budget, but ULIA dynamic is less stable as the range grows because threshold-based selection can under-select or over-select labels. Hessian-FU and retraining exhibit weaker label inference at small ranges, yet the confidence-drop and CrossLeak signals become non-trivial at larger ranges. This

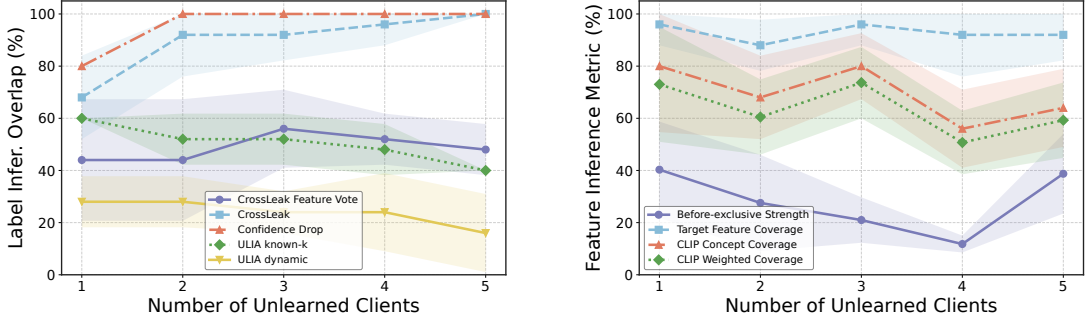


Fig. 7: CrossLeak on Different Unlearning Clients on CIFAR10

suggests that even smoother second-order unlearning and retraining can accumulate detectable class-wise shifts when the unlearning request covers more labels.

Figure 6 shows that the feature-level leakage is more persistent than label-level overlap. On CIFAR10, CrossLeak maintains high target feature coverage across ranges for FedU and Hessian-FU, with averages close to 99%, meaning that the recovered features remain concentrated on the deleted-label region even when the deleted set changes in size. CLIP concept coverage is also high, especially for FedU, showing that the recovered CrossCoder features correspond to semantically meaningful visual concepts rather than only numerical representation artifacts. Exact retraining has much lower before-exclusive strength and weaker CLIP concept coverage, which is consistent with retraining removing deleted data without creating a concentrated unlearning-update artifact.

On CIFAR100 and TinyImageNet, increasing the unlearning range makes semantic recovery harder because the label space is larger and each class contributes fewer examples. FedU still produces strong before-exclusive strength, but target feature coverage and CLIP concept coverage decline as the range grows, indicating that the deletion-sensitive features become more dispersed across fine-grained classes. Hessian-FU and retraining show weaker semantic coverage, although they still leave measurable before-exclusive feature mass. Overall, these results show that CrossLeak benefits from larger unlearning ranges for label inference, while sensitive-feature inference remains informative even when the semantic evidence becomes more diffuse.

2) *Different Unlearning Clients*: Figure 7 studies whether CrossLeak remains effective when the unlearning request is distributed over different numbers of clients. The left subfigure shows that deleted-label inference remains strong under multi-client unlearning. Confidence drop and CrossLeak both improve as more clients participate, reaching nearly perfect overlap when the deletion signal is aggregated across multiple clients. In contrast, ULIA dynamic remains much weaker, suggesting that threshold-based label selection is less reliable when the deletion evidence is spread across clients.

The right subfigure further shows that CrossLeak continues to recover deletion-sensitive features. Target feature coverage

stays high across client counts, indicating that the recovered CrossCoder features remain concentrated on the deleted-label region. CLIP concept coverage is also consistently non-trivial, which means that many recovered features can still be mapped to semantic concepts. Although the before-exclusive strength varies with the number of unlearning clients, the persistent target and concept coverage show that the paired-model difference still reveals sensitive feature-level evidence.

E. RQ3: Generalization and Application

Overall Setup. To answer RQ3, we consider two complementary scenarios. First, we test CrossLeak across different unlearning algorithms, including FedU, Hessian-FU, and exact federated retraining, on CIFAR10, CIFAR100 and TinyImageNet. The general settings are the same as in RQ1. This experiment asks whether CrossLeak can generalize to different federated unlearning algorithms.

Second, we evaluate CrossLeak in a backdoor unlearning scenario on CIFAR100. We start from a backdoored federated model with a fixed target label and then issue an unlearning request that removes the backdoored samples from the selected unlearning client. After unlearning, we compare the backdoored model before unlearning and the model after unlearning. The attacker again uses CrossLeak to identify features that are active before unlearning and suppressed after unlearning, trying to infer the backdoor triggers. We use the heatmaps to highlight the suspicious pixels that CrossLeak finds.

1) *Different Unlearning Methods*: Figure 8 evaluates sensitive feature leakage by comparing feature-overlap@1, feature-overlap@5, and feature-overlap@10 across FedU, Hessian-FU, and exact federated retraining. The three subfigures show a consistent trend: CrossLeak can recover deletion-sensitive features under all evaluated unlearning methods, and the overlap becomes clearer when more top-ranked features are considered.

On CIFAR10, the feature-overlap bars are high for FedU, Hessian-FU, and retraining, and retraining gives the strongest top-10 recovery. This means that CrossLeak is not only detecting artifacts from approximate unlearning updates; comparing the original model with an exactly retrained model can also

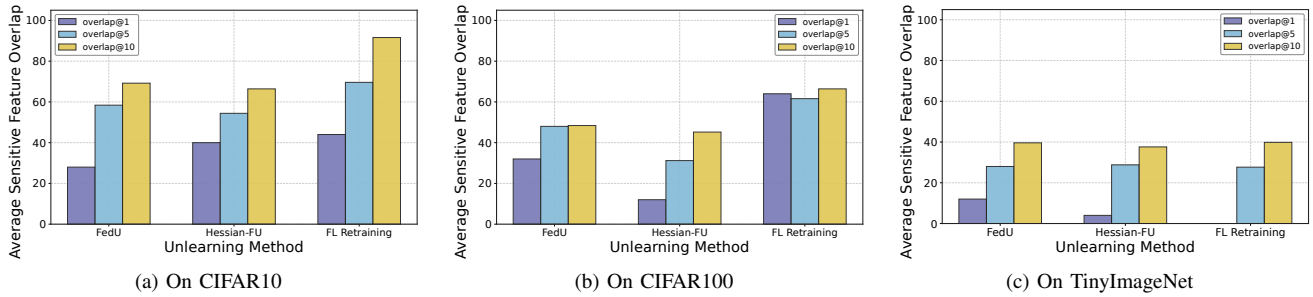


Fig. 8: Sensitive Feature inference by CrossLeak on Different Unlearning Methods



Fig. 9: Backdoor trigger inference on CIFAR100. The heatmaps visualize pixels whose CrossLeak feature scores are strongly associated with the before-unlearning backdoor behavior.

reveal the deleted feature signal. On CIFAR100, the task becomes harder because the label space is larger, but FedU and retraining still show clear feature overlap, while Hessian-FU is weaker in the top-ranked positions. On TinyImageNet, the bars are lower overall, which is expected for a larger and more fine-grained dataset, but the top-5 and top-10 overlap remains visible for all methods.

2) *Backdoor Trigger Inference*: Figure 9 visualizes CrossLeak’s trigger-related evidence after backdoor unlearning. Before unlearning, the backdoored model has a high attack success rate on triggered samples, while after unlearning the backdoor accuracy drops close to random behavior. In our multi-seed evaluation, the backdoor accuracy decreases from about 93.0% before unlearning to about 1.5% after unlearning, while the clean accuracy remains almost unchanged. This confirms that the unlearning procedure removes the functional backdoor behavior without simply destroying the clean classifier.

CrossLeak still detects a strong before-exclusive signal on backdoored probe samples. The recovered features have a high before-exclusive fraction, around 96%, and a high before-exclusive strength ratio, around 98%, showing that the trigger-related evidence is concentrated in features that are active in the backdoored model and suppressed after unlearning. This is the key signal used by CrossLeak to infer that the unlearning request affects a backdoor-related concept rather than only ordinary class information.

The heatmaps in Figure 9 provide qualitative evidence

for this inference. The highlighted regions often concentrate around visually suspicious or high-response areas of the triggered images, including regions close to the patch location in several examples. Although the heatmaps are not perfectly localized for every image, CrossLeak can reveal that the removed information is associated with a backdoor trigger and can highlight suspicious image regions.

VI. SUMMARY AND FUTURE WORK

In this paper, we studied privacy leakage in secure-aggregation federated unlearning. We showed that even when secure aggregation hides client-level updates and gradients, the original and unlearned aggregated models can still expose information about the unlearning request. To investigate this risk, we proposed CrossLeak, a post-unlearning attack that compares paired model representations before and after unlearning. By training a sparse CrossCoder on public probe samples, CrossLeak separates shared model behavior from unlearning-sensitive features, enabling both deleted-label inference and deletion-sensitive feature inference.

Our experiments on representative federated unlearning methods and datasets show that current unlearning algorithms can leak deleted labels and sensitive visual concepts under this restricted model-comparison threat model. These results suggest that secure aggregation alone is insufficient to hide the privacy signal introduced by forgetting. Future work includes developing defenses that reduce before-after representation leakage, designing verification methods that account for leakage risks, and extending the analysis to larger models, more realistic federated deployments, and stronger privacy mechanisms such as differential privacy.

REFERENCES

- [1] T. Li, A. K. Sahu, A. Talwalkar, and V. Smith, “Federated learning: Challenges, methods, and future directions,” *IEEE signal processing magazine*, vol. 37, no. 3, pp. 50–60, 2020.
- [2] B. Salehi, J. Gu, D. Roy, and K. Chowdhury, “Flash: Federated learning for automated selection of high-band mmwave sectors,” in *IEEE INFOCOM 2022-IEEE Conference on Computer Communications*. IEEE, 2022, pp. 1719–1728.
- [3] C. Zhang, S. Dang, B. Shihada, and M.-S. Alouini, “Dual attention-based federated learning for wireless traffic prediction,” in *IEEE INFOCOM 2021-IEEE conference on computer communications*. IEEE, 2021, pp. 1–10.

- [4] J. Wang, S. Guo, X. Xie, and H. Qi, "Federated unlearning via class-discriminative pruning," in *Proceedings of the ACM web conference 2022*, 2022, pp. 622–632.
- [5] Y. Liu, L. Xu, X. Yuan, C. Wang, and B. Li, "The right to be forgotten in federated learning: An efficient realization with rapid retraining," in *IEEE INFOCOM 2022-IEEE conference on computer communications*. IEEE, 2022, pp. 1749–1758.
- [6] W. Wang, C. Zhang, Z. Tian, and S. Yu, "Fedu: Federated unlearning via user-side influence approximation forgetting," *IEEE Transactions on Dependable and Secure Computing*, vol. 22, no. 3, pp. 2550–2562, 2024.
- [7] H. Hu, S. Wang, T. Dong, and M. Xue, "Learn what you want to unlearn: Unlearning inversion attacks against machine unlearning," in *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2024, pp. 3257–3275.
- [8] W. Wang, X. Tang, Y. Wang, Y. Lin, T. Zhang, M. Shen, D. Niyato, and L. Zhu, "Label inference attacks against federated unlearning," in *International Conference on Knowledge Science, Engineering and Management*. Springer, 2025, pp. 1–16.
- [9] M. Chen, Z. Zhang, T. Wang, M. Backes, M. Humbert, and Y. Zhang, "When machine unlearning jeopardizes privacy," in *Proceedings of the 2021 ACM SIGSAC conference on computer and communications security*, 2021, pp. 896–911.
- [10] L. Zhou, Y. Zhu, and R. Liu, "Model inversion attack against federated unlearning," *IEEE Transactions on Information Forensics and Security*, 2026.
- [11] J. So, R. E. Ali, B. Güler, J. Jiao, and A. S. Avestimehr, "Securing secure aggregation: Mitigating multi-round privacy leakage in federated learning," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 8, 2023, pp. 9864–9873.
- [12] K. Bonawitz, V. Ivanov, B. Kreuter, A. Marcedone, H. B. McMahan, S. Patel, D. Ramage, A. Segal, and K. Seth, "Practical secure aggregation for privacy-preserving machine learning," in *proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, 2017, pp. 1175–1191.
- [13] Z. Guan, Y. Zhao, Z. Wan, and W. Wang, "Input integrity and authentic results: Towards trustworthy aggregation in federated learning," in *IEEE INFOCOM 2025-IEEE Conference on Computer Communications*. IEEE, 2025, pp. 1–10.
- [14] A. M. Kassem, T. Jiralerspong, N. Rostamzadeh, and G. Farnadi, "Delta-crosscoder: Robust crosscoder in narrow fine-tuning regimes," in *ICLR 2026 Workshop on Principled Design for Trustworthy AI-Interpretability, Robustness, and Safety across Modalities*.
- [15] M. Pach, S. Karthik, Q. Bouniot, S. Belongie, and Z. Akata, "Sparse autoencoders learn monosemantic features in vision-language models," *Advances in Neural Information Processing Systems*, vol. 38, pp. 95 706–95 742, 2026.
- [16] J. Minder, C. Dumas, C. Juang, B. Chughtai, and N. Nanda, "Overcoming sparsity artifacts in crosscoders to interpret chat-tuning," *Advances in Neural Information Processing Systems*, vol. 38, pp. 106 423–106 474, 2026.
- [17] L. Bourtole, V. Chandrasekaran, C. A. Choquette-Choo, H. Jia, A. Travers, B. Zhang, D. Lie, and N. Papernot, "Machine unlearning," in *2021 IEEE symposium on security and privacy (SP)*. IEEE, 2021, pp. 141–159.
- [18] W. Wang, C. Zhang, Z. Tian, and S. Yu, "Machine unlearning via representation forgetting with parameter self-sharing," *IEEE Transactions on Information Forensics and Security*, vol. 19, pp. 1099–1111, 2023.
- [19] Q. P. Nguyen, B. K. H. Low, and P. Jaillet, "Variational bayesian unlearning," *Advances in Neural Information Processing Systems*, vol. 33, pp. 16 025–16 036, 2020.
- [20] N. Ding, E. Wei, and R. Berry, "Strategic data revocation in federated unlearning," in *IEEE INFOCOM 2024-IEEE Conference on Computer Communications*. IEEE, 2024, pp. 1151–1160.
- [21] A. Halimi, S. Kadhe, A. Rawat, and N. Baracaldo, "Federated unlearning: How to efficiently erase a client in fl?" *arXiv preprint arXiv:2207.05521*, 2022.
- [22] N. Su and B. Li, "Asynchronous federated unlearning," in *IEEE INFOCOM 2023-IEEE conference on computer communications*. IEEE, 2023, pp. 1–10.
- [23] X. Gao, X. Ma, J. Wang, Y. Sun, B. Li, S. Ji, P. Cheng, and J. Chen, "Verifi: Towards verifiable federated unlearning," *IEEE Transactions on Dependable and Secure Computing*, vol. 21, no. 6, pp. 5720–5736, 2024.
- [24] F. Wang, B. Li, and B. Li, "Federated unlearning and its privacy threats," *IEEE Network*, vol. 38, no. 2, pp. 294–300, 2023.
- [25] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Artificial intelligence and statistics*. Pmlr, 2017, pp. 1273–1282.
- [26] A. Li, L. Zhang, J. Tan, Y. Qin, J. Wang, and X.-Y. Li, "Sample-level data selection for federated learning," in *IEEE INFOCOM 2021-IEEE conference on computer communications*. IEEE, 2021, pp. 1–10.
- [27] P. Kairouz and H. B. McMahan, "Advances and open problems in federated learning," *Foundations and trends in machine learning*, vol. 14, no. 1-2, pp. 1–210, 2021.
- [28] A. Krizhevsky, G. Hinton *et al.*, "Learning multiple layers of features from tiny images," 2009.
- [29] Y. Le and X. Yang, "Tiny imagenet visual recognition challenge," *CS 231N*, vol. 7, no. 7, p. 3, 2015.